

Operating System

1.Elaborate on operating system services?

- Provide a set of **services** that act as an interface between **users/applications and the computer hardware**
- These services make systems **convenient to use, efficient, reliable, and secure.**

1. User Interface (UI) - Interactions between the user and computer

2. Program Execution - Running application

3. Process Management - The OS manages **multiple processes** efficiently

4. Memory Management - The OS manages the system's main memory (RAM).

5. File System Management - The OS organizes and manages data stored on disks. (Creating, deleting, reading, and writing files)

6. I/O Device Management - Controls input and output devices through **device drivers.**

7. Communication Services - Enables communication between processes. (Inter-process communication, Network communication:)

8. Error Detection and Handling - The OS constantly monitors the system for errors. (Hardware failures, I/O errors)

9. Resource Allocation - The OS decides how system resources are distributed.(CPU time, Memory)

10. Accounting - The OS tracks resource usage. (CPU usage time, Memory consumption)

11. Protection and Security - The OS safeguards data and system resources.(User authentication (login/passwords)).

OS Service	Main Function
User Interface	Interaction with users
Program Execution	Running applications
Process Management	Handling multitasking
Memory Management	Efficient RAM usage
File System	Data storage organization
I/O Management	Device control
Communication	Process and network interaction
Error Handling	System stability
Resource Allocation	Fair resource usage
Accounting	Usage tracking
Security	Protection and access control

7. Write detailed notes on file system organization/file operations?

File System Organization

- A file system is a method used by the operating system to store, organize, manage, and retrieve data
- Use secondary storage devices such as hard disks, SSDs, or flash drives.
- File system organization defines **how files are named, stored, accessed, and protected**.

1. Objectives of File System Organization

- Efficient storage and retrieval of data
- Logical organization of files
- Protection and security of data
- Support for sharing and concurrency
- Minimization of storage fragmentation

2. File Concept

A **file** is a collection of related information stored on secondary storage.

File Attributes

Each file has associated metadata:

- **Name** – human-readable identifier
- **Identifier** – unique internal number
- **Type** – text, binary, executable, etc.
- **Location** – pointer to storage blocks
- **Size** – current and maximum size
- **Protection** – access rights
- **Time & date** – creation, access, modification

3. Directory Structure

A directory is a special file that contains information about other files.

- Single-Level Directory
- Two-Level Directory
- Tree-Structured Directory
- Acyclic Graph Directory
- General Graph Directory

4. File Allocation Methods

- Defines how disk blocks are assigned to files.
 - Contiguous Allocation
 - Linked Allocation
 - Indexed Allocation

5. Free Space Management

The OS tracks unused disk blocks using:

- Bitmaps
- Linked lists
- Grouping
- Counting

File Operations

File operations are actions performed by the operating system on files to manipulate data and metadata.

1. Create

- Allocates space for a new file
- Adds an entry in the directory
- Initializes file attributes

2. Open

- Loads file metadata into memory
- Returns a **file descriptor/handle**
- Sets file pointer to beginning

3. Read

- Transfers data from file to memory
- Uses file pointer to track position
- Advances pointer after reading

4. Write

- Transfers data from memory to file
- May allocate new disk blocks
- Updates file size and timestamps

5. Append

- Writes data at the **end of the file**
- Prevents overwriting existing data

6. Seek (Reposition)

- Moves file pointer to a specific location
- Enables random access

7. Close

- Releases file descriptor
- Writes modified metadata back to disk

8. Delete

- Removes directory entry
- Frees allocated disk blocks
- Updates the free space structure

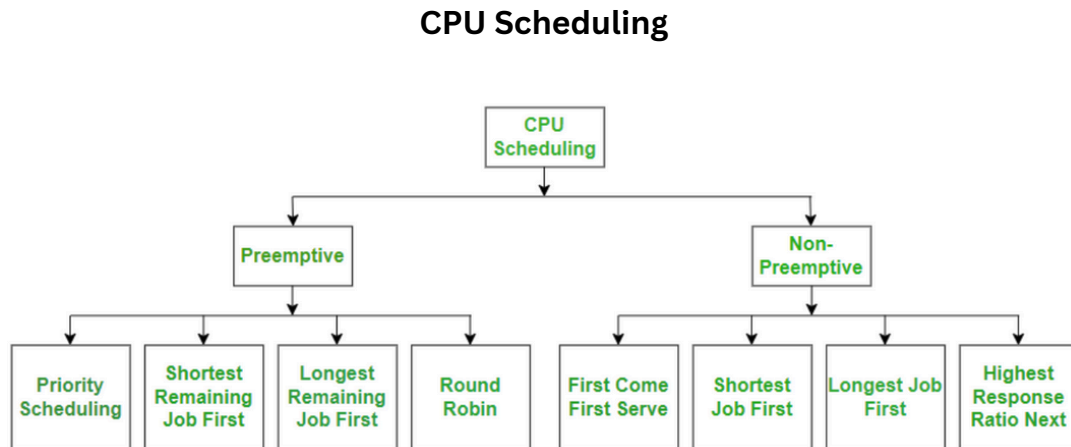
9. Truncate

- Removes file contents but keeps attributes
- File size becomes zero

10. Rename

- Changes the file name without altering the contents

2. Explain CPU scheduling / any one scheduling algorithm.



- It decides **which process gets the CPU, when, and for how long**
- The CPU is a shared resource.
- Multiple processes compete for CPU time.
- Proper scheduling improves:
 - CPU utilization
 - System responsiveness
 - Throughput
 - Fairness among processes

Subtopics of CPU Scheduling

1. Process States

- New
- Ready
- Running
- Waiting (Blocked)
- Terminated

2. Scheduling Queues

- **Ready Queue** – processes waiting for CPU
- **Waiting Queue** – processes waiting for I/O
- **Job Queue** – all processes in the system

3. Types of CPU Scheduling

- **Preemptive Scheduling**
 - CPU can be **taken away from a running process**

- **Non-Preemptive Scheduling**
 - Process **keeps CPU until it finishes** or blocks

4. CPU Scheduling Criteria

- CPU Utilization
- Throughput
- Turnaround Time
- Waiting Time
- Response Time

5. Scheduling Algorithms

- First Come First Serve (FCFS)
- Shortest Job First (SJF)
- Shortest Remaining Time First (SRTF)
- Priority Scheduling
- Round Robin (RR)
- Multilevel Queue Scheduling
- Multilevel Feedback Queue Scheduling

6. Dispatcher

- Module that gives control of CPU to the selected process
 - Context switching
 - Switching to user mode
 - Jumping to correct instruction

One Scheduling Algorithm: First Come First Serve (FCFS)

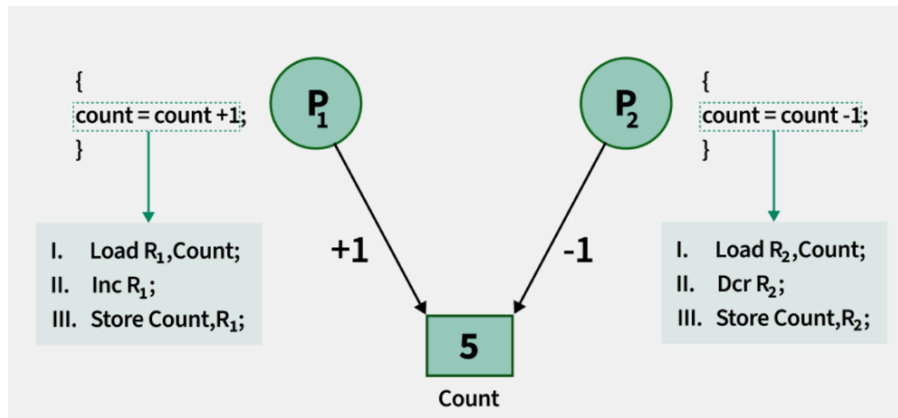
- **FCFS** is the **simplest CPU scheduling algorithm**.
- Processes are executed **in the order they arrive** in the ready queue.
 - Process that arrives first gets the CPU first.
 - Once a process starts execution, it **runs until completion**.
 - It is a **non-preemptive** algorithm.

Process	Arrival Time	Burst Time
P1	0	5
P2	1	3
P3	2	2

Execution Order:

P1 → P2 → P3

3. Discuss process synchronization in detail.



- **Process synchronization** is a mechanism used by an **OS**.
- To manage the execution of **multiple processes that share common resources**.
- Ensuring **data consistency** and **correct program execution**.

Process Synchronization is Needed for:

- Race conditions
- Inconsistent data
- Incorrect output

Problems Without Synchronization:

- **Race Condition**
 - Occurs when multiple processes access shared data simultaneously and the final result depends on the order of execution.
- **Deadlock**
 - A situation where processes wait indefinitely for resources held by each other.
- **Starvation**
 - A process waits forever because it never gets CPU or resource access.

1. Mutex (Mutual Exclusion) - Only one process/thread can hold the lock at a time

2. Semaphore - **signaling mechanism** used to control access to shared resources.

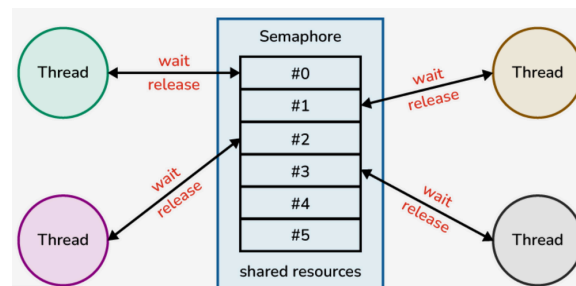
3. Monitors -

- Shared data
- Procedures
- Synchronization

4. Spinlock

- Process keeps checking (spinning) until lock is available
 - Used in multiprocessor systems
-

4. Illustrate semaphores and synchronization methods.



- A semaphore is a synchronization tool used in an Operating System to **control access to shared resources by multiple processes** or threads.
- It uses an **integer variable** and two **atomic operations** to ensure safe execution.

Semaphore Operations

- Wait() / P()
 - Decrements the semaphore value
 - If value becomes negative, the process is **blocked**
- Signal() / V()
 - Increments the semaphore value
 - Wakes up a waiting process (if any)

Types of Semaphores

1. Binary Semaphore

- Value: **0 or 1**
- Works like a **mutex lock**
- Allows only **one process** in the critical section

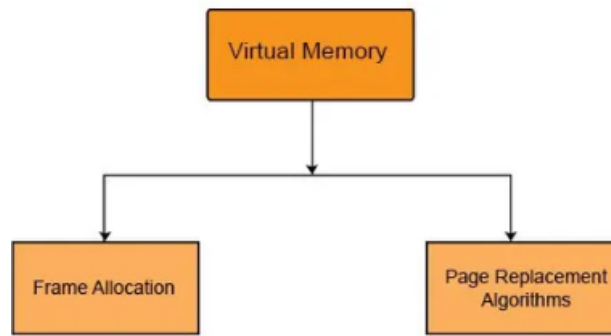
Example Use: Mutual exclusion

2. Counting Semaphore

- Value: **0 to n**
- Allows **multiple processes** to access a limited number of resources

Example Use: Printer pool

5. Explain page replacement and its algorithms.

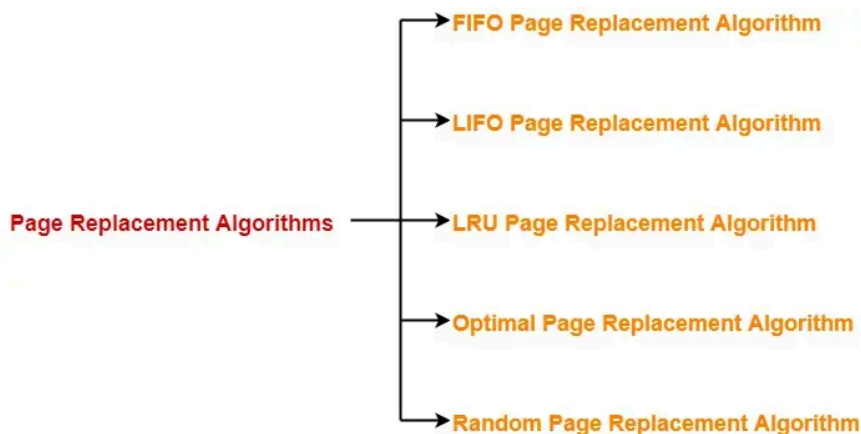


Page Replacement – Definition

- **Page replacement** is a **memory management technique** used in an Operating System when a **page fault** occurs and the **main memory (RAM) is full**.
- The OS must decide **which page to remove (replace)** to load the required page from secondary memory.

Page Replacement Is Needed

- Main memory has **limited capacity**
- Programs are larger than available RAM
- Efficient page replacement:
 - Reduces page faults
 - Improves system performance
 - Optimizes memory utilization



1. First In First Out (FIFO)

The **oldest page** in memory is replaced first.

2. Optimal Page Replacement

Replace the page that **will not be used for the longest time in the future**.

3. Least Recently Used (LRU)

- Replace the page that **has not been used for the longest time in the past**.

4. Least Frequently Used (LFU)

- Replace the page with the **lowest access count**.

5. Most Recently Used (MRU)

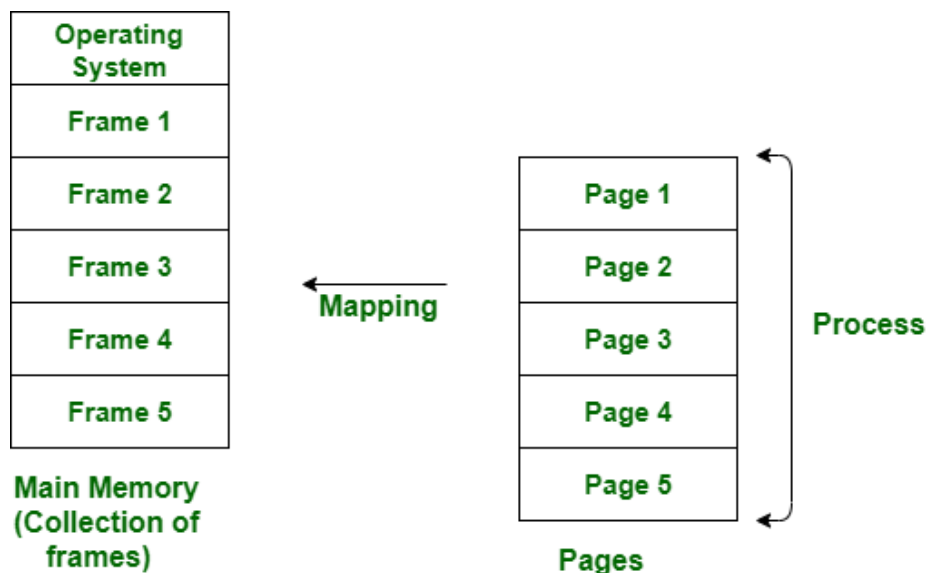
- Replace the **most recently used page**.

6. Discuss paging and segmentation?

- **Paging** and **Segmentation** are **memory management techniques** used by an Operating System
- To efficiently manage **main memory** and **support virtual memory**.

Paging

- **Logical (virtual) memory** into fixed-size blocks called **pages**
- **Physical memory** into fixed-size blocks called **frames**



Paging

Pages are mapped to frames using a **page table**.

A **logical address** consists:

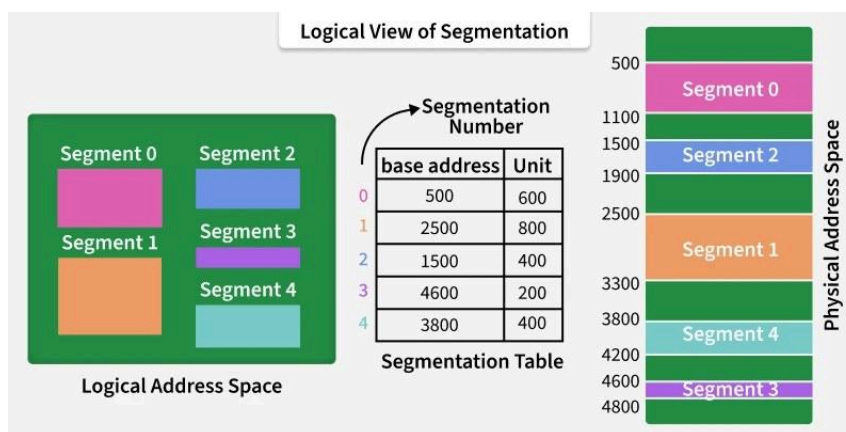
- **Page Number** – index into page table
- **Offset** – position within the page

The page table provides the frame number; **frame number + offset gives the physical address**.

Advantages of Paging

- Eliminates external fragmentation
- Simple memory allocation
- Supports virtual memory efficiently

Segmentation



- **Segmentation** divides memory into **variable-sized logical units** called **segments**, based on program structure such as
 - **Code** segment
 - **Data** segment
 - **Stack** segment
- Each segment has a **base address** and a **limit**.

Address Translation in Segmentation

- Segment Number
- Offset

The offset is checked against the segment limit, then added to the base address to form the physical address.

Advantages of Segmentation

- Matches programmer's logical view
- No internal fragmentation
- Easier protection and sharing

Feature	Paging	Segmentation
Block Size	Fixed	Variable
Programmer View	Not visible	Visible
Fragmentation	Internal	External
Memory Allocation	Simple	Complex
Protection	Page level	Segment level

8. Explain process control and context switching in UNIX?

In UNIX, process control and context switching are fundamental mechanisms that enable **multitasking, process management, and efficient CPU utilization**.

Process Control in UNIX

- **Process control** refers to the creation, execution, scheduling, synchronization, and termination of processes in a UNIX OS.
- Each process in UNIX is uniquely identified by a **Process ID (PID)** and is managed by the kernel.

Process Creation - `fork()`, `exec()`

Process Termination - `exit()`

Process States in UNIX -

- **Running** – executing on CPU
- **Ready** – waiting for CPU
- **Sleeping (Blocked)** – waiting for I/O or event
- **Stopped** – suspended
- **Zombie** – terminated but entry exists in process table

Process Control Block (PCB)

- PID
- Process state
- Program counter
- CPU registers
- Scheduling information
- Memory management information
- Open files

Context Switching in UNIX

Context switching is the process of **saving the state of the currently running process** and **loading the state of another process** so the CPU can switch execution.

When Context Switching Occurs

- Time slice expires
- Process performs I/O operation
- Higher priority process becomes ready
- Interrupt or system call occurs

Steps in Context Switching

1. Save current process context (registers, PC, stack pointer) into PCB

2. Update process state
3. Select next process from ready queue
4. Load new process context from PCB
5. Resume execution of new process

Overhead of Context Switching

- No useful work during switch
- Consumes CPU time
- Frequent switching reduces system performance

Relationship Between Process Control and Context Switching

- Process control **decides what process runs**
 - Context switching **enables the actual switch**
 - Both are handled by the **UNIX kernel**
-

9. Explain security issues in file systems?

Major Security Issues in File Systems

1. Unauthorized Access

2. Improper Access Control - Incorrect permission settings, Users granted more privileges than required

3. Data Tampering - Unauthorized modification of file contents

4. Data Loss and Destruction - Files may be deleted intentionally or accidentally

5. Malware and Virus Attacks -

6. Insecure File Sharing - Shared files accessible to unintended users

7. Lack of Encryption - Files stored in plain text can be easily read

8. Weak Backup and Recovery - Absence of proper backups, Data cannot be restored after failure or attack

File System Security Mechanisms

1. File Permissions - Control **read (r)**, **write (w)**

2. Access Control Lists (ACLs) -

- Provide fine-grained access control
- Specify permissions for individual users or groups

3. Authentication - Verifies user identity before granting access

4. Encryption - Ensures data remains unreadable without a key

5. Backup and Recovery - Regular backups prevent permanent data loss

6. Auditing and Logging -

- Records file access and changes
 - Helps detect unauthorized activity
-

10. Explain Inter-process communication in detail.

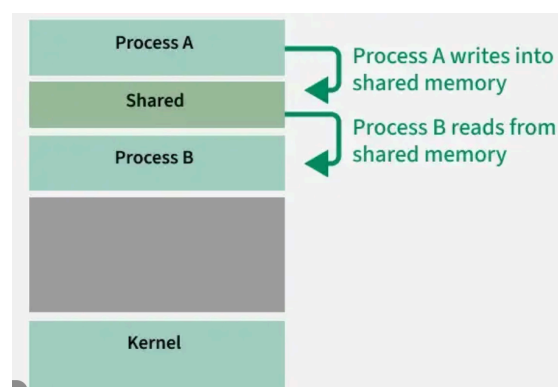
Inter-Process Communication (IPC)

- Mechanisms provided by an operating system
- That allow **processes to communicate with each other and synchronize their actions.**
- **Information sharing** between processes
- **Computation speed-up** through parallel execution
- **Modularity** (dividing tasks into cooperating processes)
- **Convenience** (multiple applications working together)

Models

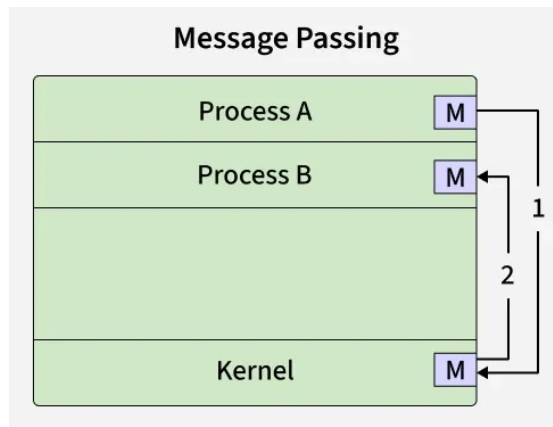
1. Shared Memory Model

- A region of memory is shared between cooperating processes.
- Processes read and write to this shared memory.
- Synchronization is required to avoid race conditions.



2. Message Passing Model

- Processes communicate by sending and receiving messages.
- Useful in distributed systems.



Methods of Inter-Process Communication

1. Pipes

- Unidirectional communication between related processes.
- Common in UNIX systems.

2. Named Pipes (FIFOs)

- Allow communication between unrelated processes.

3. Message Queues

- **Messages** are stored in a queue.
- Processes communicate asynchronously.

4. Shared Memory

- Fastest IPC method.

5. Semaphores

- Used for synchronization, not data transfer.
- Prevent race conditions.

11. Elaborate reliability improvements.

- Refers to its ability to **function correctly, consistently, and continuously**, even in the presence of hardware failures, software errors, or unexpected conditions.
- Reduce system crashes and failures
- Prevent data loss and corruption
- Ensure continuous system availability
- Increase user trust and system lifespan

Techniques for Reliability Improvements

1. Fault Tolerance

Fault tolerance enables the system to continue operating even when a failure occurs.

2. Error Detection and Correction

Helps prevent data corruption during storage and transmission

3. Process Isolation and Protection

- Each process runs in its own **address space**
- Prevents one faulty process from affecting others

4. Recovery Mechanisms

Checkpointing, Rollback recovery, System reboot and restart services

5. Backup and Restore

Regular data backups (incremental or full)

Enables recovery from hardware failures or accidental deletion

15. Explain Deadlock

Aspect	Prevention	Avoidance	Detection
Deadlock Occurrence	Impossible	Avoided	Allowed
Resource Utilization	Low	Medium	High
Complexity	Low	High	Medium
Information Required	No	Yes	No

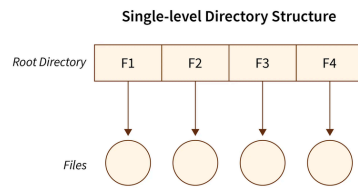
15. Explain directory structure organization.

- Organizes files into directories
- so that files can be **stored, located, and managed efficiently**
- A directory is a special file that contains information about other files such as **file name, location, size, and permissions.**
- **Organize files** systematically
- Allow **easy file searching** and access

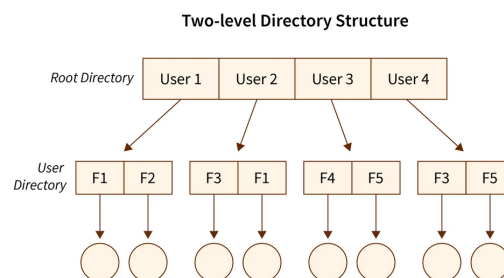
- **Improve file system** efficiency

Types of Directory Structures

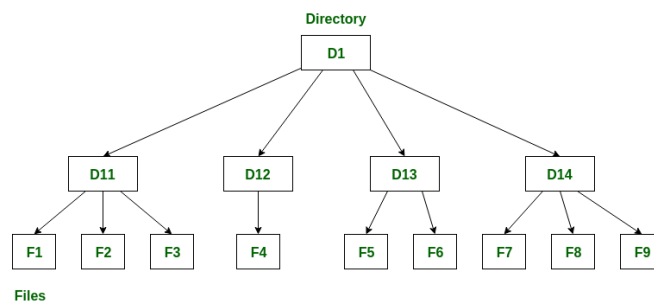
1. Single-Level Directory



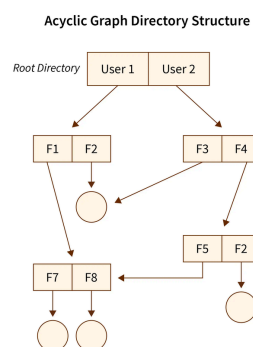
2. Two-Level Directory



3. Tree-Structured Directory

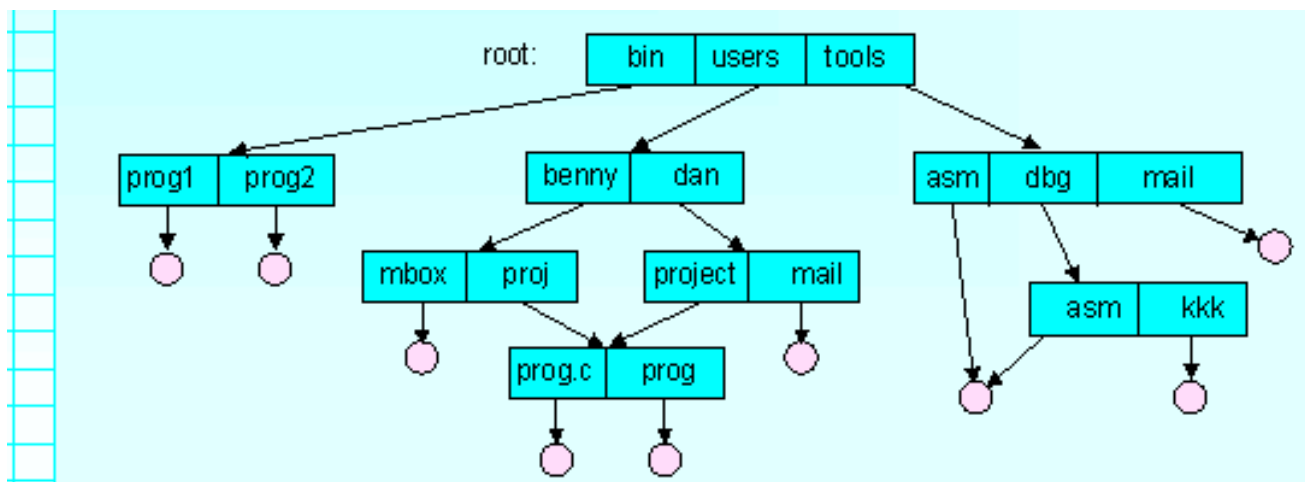


4. Acyclic Graph Directory



5. General Graph Directory

- Allows cycles in directory structure.
- Requires garbage collection to avoid infinite loops.



16. Explain storage hierarchy.

- Structured arrangement of different storage devices in a computer system
- Based on **speed, cost, capacity, and access time**.
- It helps achieve **high performance at low cost**

Levels of Storage Hierarchy

1. Registers

- Located inside the CPU
- Fastest storage
- Very small capacity

2. Cache Memory

- Very high-speed memory between CPU and main memory
- Stores frequently accessed data

3. Main Memory (Primary Memory)

Usually **RAM**

4. Secondary Storage

- Hard Disk Drives (HDD)
- Solid State Drives (SSD)

5. Tertiary Storage

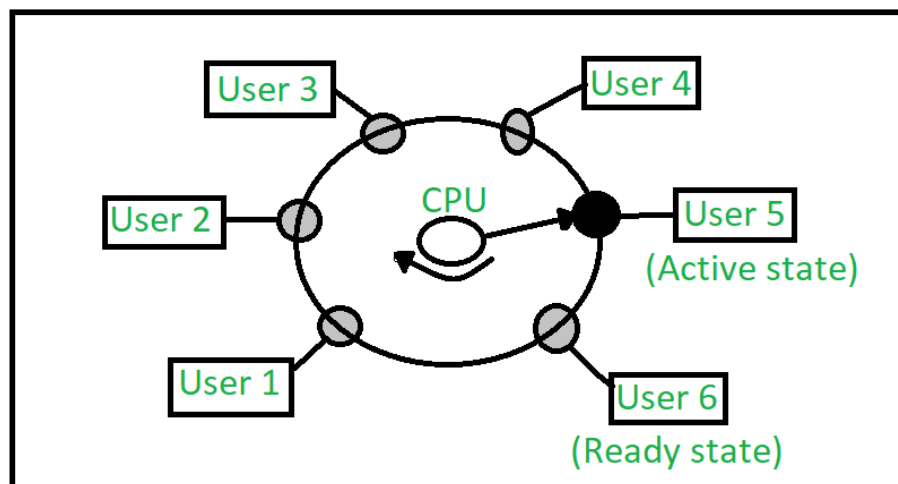
- Used for backup
- Very large capacity

Level	Speed	Cost	Capacity
Registers	Very High	Very High	Very Low
Cache	High	High	Low
Main Memory	Medium	Medium	Medium
Secondary Storage	Low	Low	High
Tertiary Storage	Very Low	Very Low	Very High

5 MARK

1. Write short notes on time sharing system.

- A **time sharing system** is an operating system technique in which **multiple users or processes share the CPU simultaneously** by
- Dividing CPU time into small units called **time slices**
- Each process gets the **CPU for a short duration**, creating the illusion that all processes are executing at the same time.



Features of Time Sharing System

- Multi-user support
- Rapid context switching
- Preemptive CPU scheduling
- Interactive response
- Fair CPU allocation

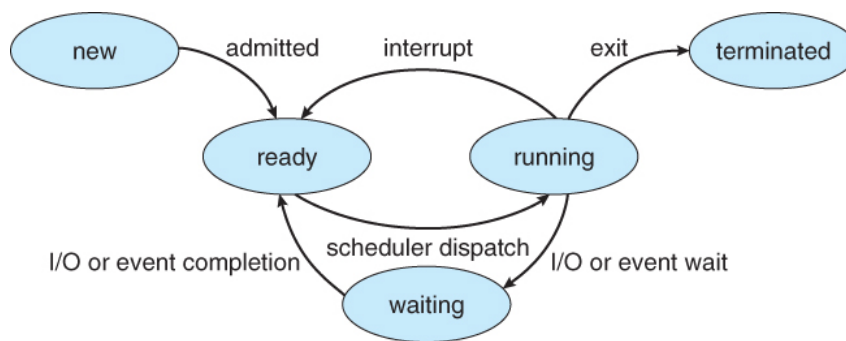
2. Explain process concept / concurrent process.

Process Concept

- A process is a **execution of program**

Components of a Process

- **Program code** (text section)
- **Program counter** (address of next instruction)
- **CPU registers**
- **Stack** (function calls, local variables)
- **Data section** (global and static variables)
- **Resources** (files, I/O devices, memory)

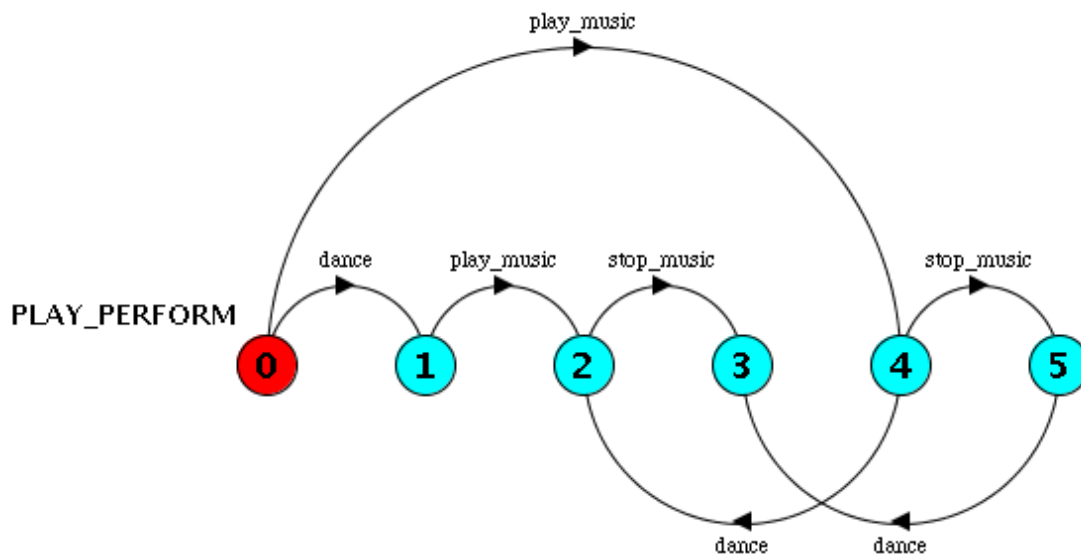


Concurrent Process

- **Concurrent processes** are **multiple processes that are in progress at the same time**

Issues in Concurrent Processes

- Race conditions
- Critical section problem
- Deadlock
- Starvation



7. Explain design principles of UNIX / Windows OS.

- Operating systems are designed based on specific principles to ensure **efficiency, reliability, security, and usability**.

UNIX OS

- 1. Simplicity** Small, simple programs
- 2. Modularity** Programs can be combined using pipes
- 3. Portability** Written mainly in the C programming language
- 4. Everything Is a File** Devices, files, and directories are treated uniformly
- 5. Multiuser and Multitasking** Supports multiple users and multiple processes simultaneously
- 6. Hierarchical File System** Tree-structured directory organization
- 7. Security and Protection**
 - User permissions (read, write, execute)
 - Ownership and access control

Windows OS

- 1. User-Friendly Graphical Interface** - Rich GUI with windows, icons, menus, and pointers
- 2. Layered and Modular Architecture** - Divides system into user mode and kernel mode
- 3. Object-Oriented Design** - OS components represented as objects

4. Preemptive Multitasking - Supports efficient multitasking using threads

5. Hardware Abstraction - Improves portability across hardware

6. Security - User authentication, encryption

7. Backward Compatibility - Supports older applications and APIs

Aspect	UNIX	Windows
Design	Simplicity	Usability
Interface	Command-line based	Graphical
Portability	High	Moderate
Security	Strong permissions	Normal
File Handling	Everything is a file	Object-based

8. Give short notes on system call / dual mode operation

System Call

- A system call is a mechanism that allows a user program to request services from the operating system kernel.
- Since user programs cannot directly access hardware, system calls act as a **controlled interface** between user space and kernel.
- Provides safe and secure access to OS services.

Functions of System Calls

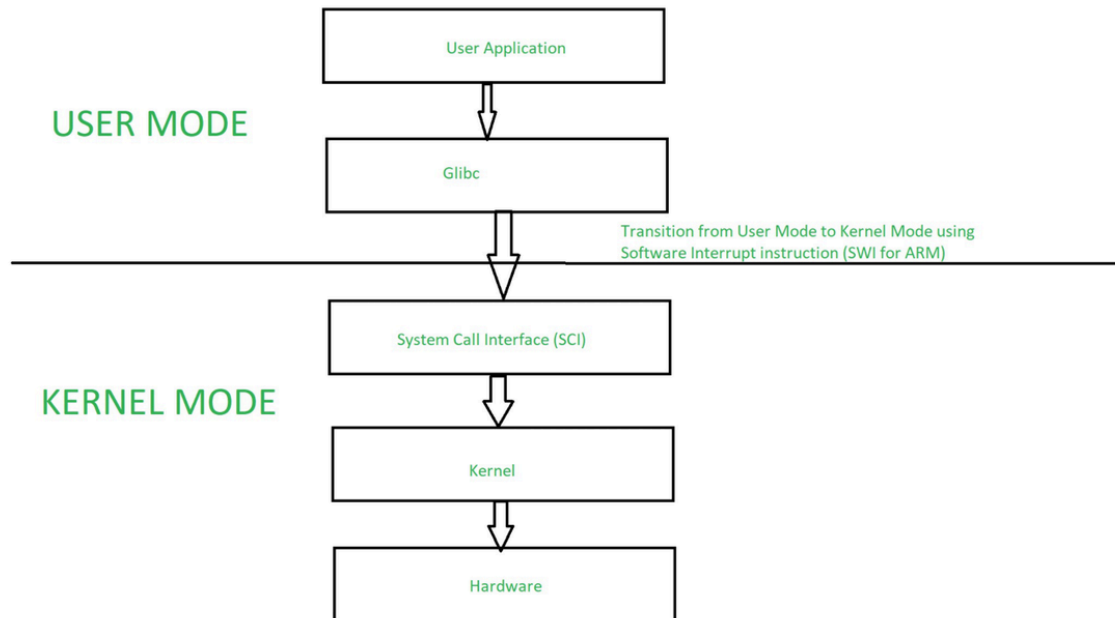
- Process control (create, terminate)
- File management (open, read, write, close)
- Device management
- Information maintenance
- Communication

Examples

- `fork()` – create a process
- `read()` / `write()` – file I/O
- `exec()` – execute a program
- `exit()` – terminate a process

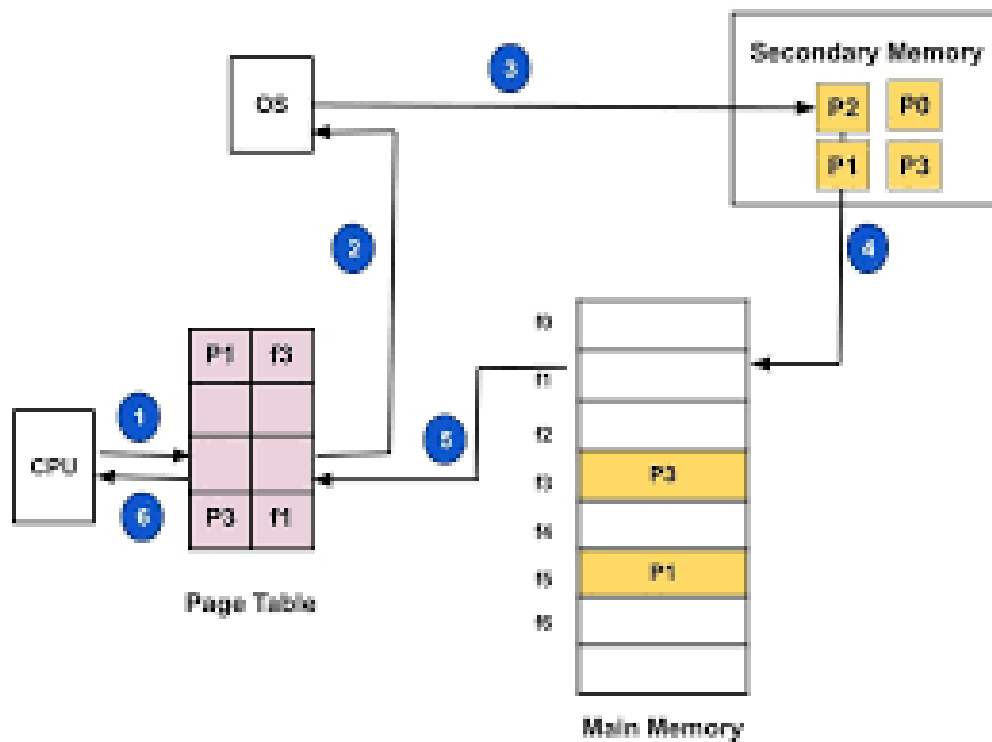
Dual Mode Operation

1. User Mode -
 - a. Executes user programs
 - b. Restricted access (cannot access hardware directly)
2. Kernel Mode
 - a. Executes OS kernel code
 - b. Full access to hardware and system resources



9. Describe demand paging.

- Pages of a process are loaded into main memory only when they are actually needed
- Rather than loading the entire process at once
- It is a key feature of **virtual memory systems**.

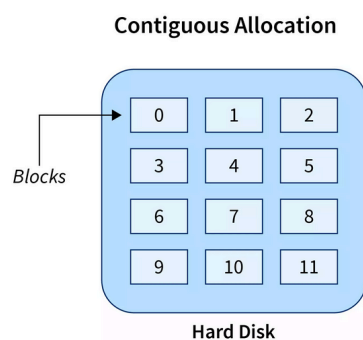


10. Explain allocation methods / disk space allocation.

File Allocation Methods

Disk space allocation refers to the methods used by an operating system to **allocate disk blocks to files** so that files can be stored, accessed, and managed efficiently.

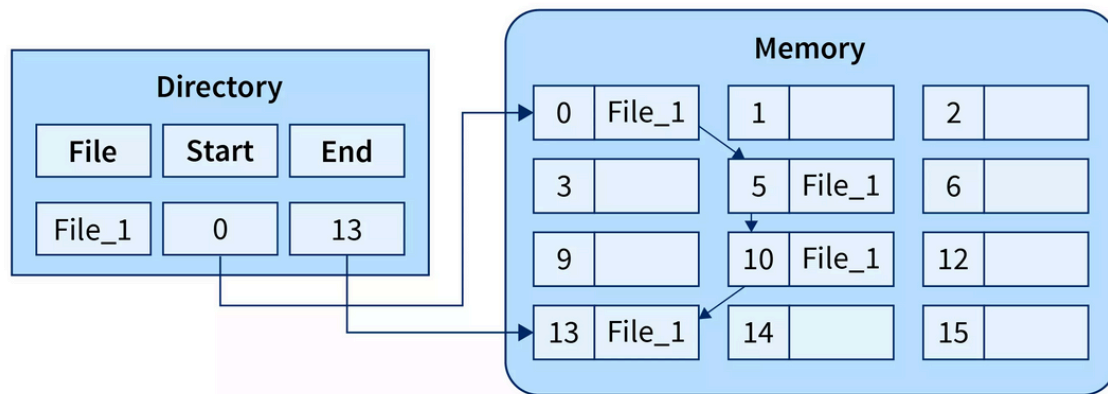
1. Contiguous Allocation



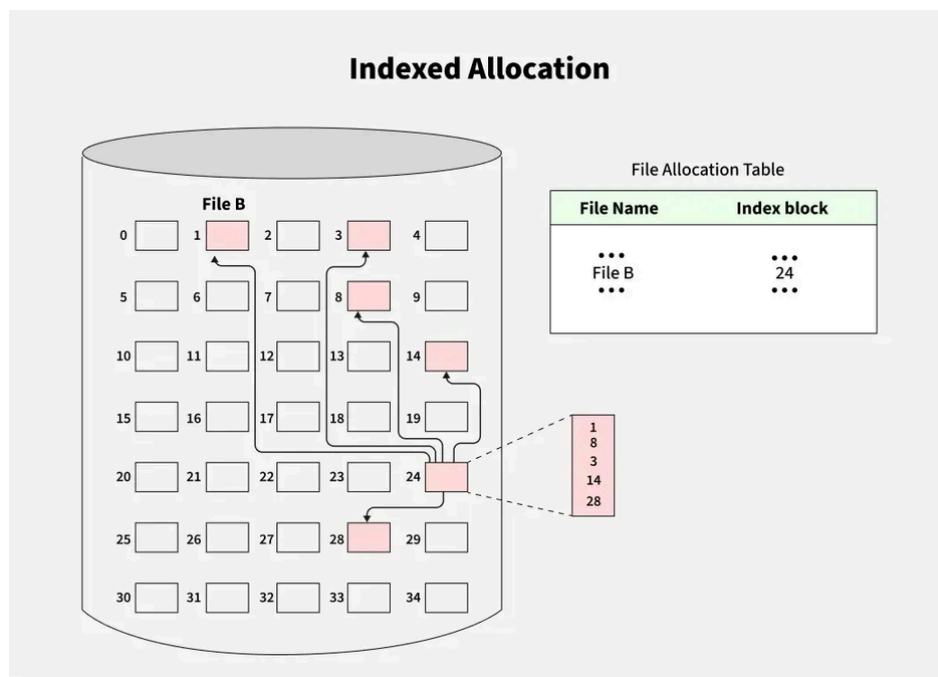
File Name	Start	Length	Allocated Blocks
file1.txt	0	4	0,1,2,3
sun.jpg	5	3	5,6,7
mov.mp4	9	3	9,10,11

Directory

2. Linked Allocation



3. Indexed Allocation



11. Describe consistency semantics.

1. **Strict Consistency** - Any read returns the **most recent write** immediately.
2. **Sequential Consistency** - All processes see the **same order of updates**
3. **Causal Consistency** - Concurrent (independent) writes may be seen in different orders
4. **Weak Consistency** - Updates visible only at **synchronization points**
5. **Release Consistency** - Modern multiprocessor systems

14. LRU, FIFO, Clock replacement – short notes.

1. **FIFO (First-In First-Out) Page Replacement**
2. **LRU (Least Recently Used) Page Replacement** -

- Replaces the page that **has not been used for the longest time**.

3. Clock (Second-Chance) Page Replacement -

- Approximation of LRU.
 - Pages arranged in a **circular list (clock)**.
 - Uses a **reference bit**:
 - If bit = 0 → replace page
 - If bit = 1 → reset bit and give second chance
-

15. Real-time systems?

- A **real-time system** is a computer system in which **correctness depends not only on logical results but also on the time at which the results are produced**.
- In such systems, missing a deadline can lead to system failure.

Types of Real-Time Systems

1. Hard RTS

- Deadlines must **never be missed**
- Missing a deadline causes **failure**

2. Soft RTS

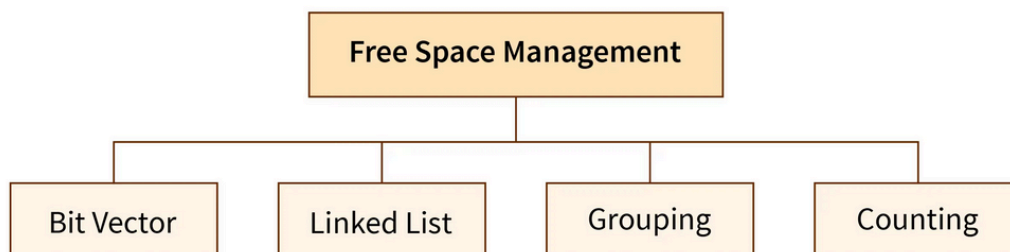
- Deadlines are important but occasional misses are tolerable
- Performance degrades but system does not fail

3. Firm RTS

- Deadlines must be met, but late results are useless
 - Occasional misses acceptable, but results discarded
-

16. Free space management survey.

- Track and manage unused disk blocks
- So they can be efficiently allocated to files when needed and reclaimed when files are deleted.



1. Bit Map (Bit Vector)

Each disk block is represented by **1 bit**:

- 0 → free block
- 1 → allocated block

2. Linked List

- All free disk blocks are linked together.

3. Grouping

- Modification of linked list method.
- First free block stores addresses of **multiple free blocks**.

4. Counting

Keeps track of **contiguous free blocks** as:

- Starting block number
 - Count of free blocks
-

17. Thrashing short explanation.

- **Thrashing** is a condition in an operating system
- The system spends **most of its time handling page faults instead of executing processes**
- Resulting in very low CPU utilization and poor performance.

Effects of Thrashing

- Excessive paging (swap in and swap out)
 - Increased disk I/O
 - Reduced system throughput
 - System slowdown
-